

Forum Web — Rapport de Projet

Madina LALAOUI & Rasheequa BAGADADSAIB

1. Répartition du travail

Madina LALAOUI — Frontend · React / Vite

- › App.jsx : état global, routing React Router, persistance via localStorage
- › ForumPage : sélection du forum, filtrage récursif par mot-clé et plage de dates, orchestration des actions
- › Message.jsx : composant récursif pour l'affichage des réponses imbriquées à profondeur arbitraire
- › AdminDashboard : validation des inscriptions en attente, promotion et rétrogradation des membres
- › ProfilPage, LoginPage, SignupPage : formulaires avec gestion des erreurs et redirections
- › utils/messages.js : fonctions pures immuables (createMessage, deleteRecursive, addReplyRecursive, toggleLikeRecursive)
- › CSS par page et icônes Lucide

Rasheequa BAGADADSAIB — Backend · Express.js / MongoDB / DevOps

- › Serveur Express : middleware CORS, JSON, utilitaire withDB() pour centraliser la gestion des erreurs
- › 8 routes utilisateurs (signup, login, liste, profil, validation, rejet, rôle) et 2 routes messages
- › Connexion MongoDB avec le driver natif — base nommée forum_ML_RB
- › Hachage bcrypt des mots de passe dès l'inscription, comparaison à la connexion
- › Logique de rôles et statuts : premier inscrit admin, comptes suivants en pending, auto-modification interdite
- › Seed automatique au premier lancement si la base est vide
- › Déploiement sur Google Cloud via une VM, avec nom de domaine et certificat HTTPS
- › Bot de scraping de news : récupère automatiquement des actualités et les poste sur le forum

2. Problèmes rencontrés

Structure récursive des messages : gérer les réponses imbriquées à profondeur arbitraire (suppression, ajout, like) a été le principal défi. Résolu avec des fonctions pures récursives et un composant Message.jsx qui s'appelle lui-même pour chaque niveau de réponse.

Synchronisation front/back : le useEffect de sauvegarde se déclenchait au premier rendu avec un tableau vide, écrasant les données en base. Corrigé avec un useRef (initialLoad) pour ignorer ce premier déclenchement.

Cas limites serveur : la route GET /users/pending devait être déclarée avant GET /users/:username pour éviter que « pending » soit capturé comme paramètre. Protection ajoutée contre l'auto-modification de rôle par un admin.

Filtrage récursif des messages : la recherche devait conserver un fil entier si au moins une réponse correspondait au filtre. Résolu avec filterMessagesRecursive qui descend dans l'arbre avant de décider d'exclure un message.

État global React : partage de setMessages entre App.jsx, ForumPage et ProfilPage via props, avec mise à jour optimiste dans le dashboard admin pour éviter des refetch inutiles.

3. Ce qui reste à faire

- › **Authentification côté serveur** : les routes sensibles (PUT /users/:username/role, /validate, /reject) ne vérifient pas si l'appelant est connecté ni s'il est admin. N'importe qui peut les appeler directement via curl. Un token JWT ou une session Express serait nécessaire.
- › **Protection du forum privé côté serveur** : GET /messages retourne tous les messages sans filtrer par forumId. La restriction au forum privé est uniquement gérée côté client dans ForumType.jsx, ce qui ne protège pas contre un appel direct à l'API.
- › **Mise à jour automatique** : les messages postés par d'autres utilisateurs n'apparaissent pas sans recharger la page. Aucun polling ni WebSocket n'est implémenté.
- › **Modification de profil** : ProfilPage.jsx ne contient aucun formulaire d'édition. L'utilisateur ne peut pas modifier son prénom, nom ou mot de passe.

4. Choix de modélisation différents du cours

- **Document unique pour les messages** : plutôt qu'un document MongoDB par message, nous stockons l'arbre complet dans un seul document `{ key: "messages", value: [...] }`. Les réponses imbriquées récursives auraient nécessité des jointures complexes avec des documents séparés. Ici, une seule opération suffit pour lire ou écrire tout l'arbre.
- **Hachage bcrypt** : non exigé explicitement en cours, mais intégré dès le départ pour ne jamais stocker de mot de passe en clair. Comparaison via `bcrypt.compare()` à la connexion.
- **Session stateless via localStorage** : pas de session côté serveur. Les données de l'utilisateur connecté sont stockées dans le localStorage du navigateur, ce qui simplifie l'architecture serveur.

Schéma logique de la base de données MongoDB

Serveur : `http://localhost:3001`

La base contient 2 collections : users et posts.

Collection : users

```
{
  "username": "string",
  "password": "string (bcrypt hash)",
  "firstName": "string",
  "lastName": "string",
  "role": "member | admin",
  "status": "pending | validated",
  "dateCreation": "date"
}
```

Collection : posts

```
{
  "key": "messages",
  "updatedAt": "date",
  "value": [{
    "id": "string",
    "auteur": "string",
    "createdAt": "string (ISO date)",
    "contenu": "string",
    "forumId": 1,
    "likes": ["string"],
    "reponses": [{
      "id": "string", "auteur": "string",
      "createdAt": "string",
      "contenu": "string",
      "likes": ["string"], "reponses": []
    }]
  }]
}
```

Routes Messages

GET /messages — Récupérer tous les messages

```
curl http://localhost:3001/messages
```

Résultat :

```
{
  "messages": [
    {
      "id": "1780084074035-3ff5oiguu48",
      "auteur": "madina",
      "createdAt": "2026-04-15T19:47:54.035Z",
      "contenu": "heey",
      "reponses": [
        {
          "id": "1780084543413-18v5j4duo4f",
          "auteur": "jade",
          "createdAt": "2026-04-15T19:55:43.413Z",
          "contenu": "helloooo",
          "reponses": [],
          "likes": []
        }
      ],
      "likes": ["jade"],
      "forumId": 1
    }
  ]
}
```

PUT /messages — Sauvegarder les messages

```
curl -s -X PUT http://localhost:3001/messages \
-H "Content-Type: application/json" \
-d '{
  "messages": [
    {
      "id": "1780084074035-3ff5oiguu48",
      "auteur": "madina",
      "createdAt": "2026-04-15T19:47:54.035Z",
      "contenu": "heey",
      "forumId": 1,
      "likes": ["jade"],
      "reponses": [
        {
          "id": "1780084543413-18v5j4duo4f",
          "auteur": "jade",
          "createdAt": "2026-04-15T19:55:43.413Z",
          "contenu": "helloooo",
          "reponses": [],
          "likes": []
        }
      ]
    }
  ]
}'
```

Résultat :

```
{"message": "Messages sauvegardés"}
```

PUT /messages — Erreur : body invalide

```
curl -s -X PUT http://localhost:3001/messages \
-H "Content-Type: application/json" \
-d '{"messages": "mauvais format"}'
```

Résultat :

```
{"message": "Format de messages invalide"}
```

Routes Utilisateurs

POST /login — Connexion valide

```
curl -X POST http://localhost:3001/login \  
-H "Content-Type: application/json" \  
-d '{"username":"jade","password":"Web789;"}'
```

Résultat :

```
{"username":"jade","role":"member",  
"firstName":"Jade","lastName":"DUPONT"}
```

POST /login — Erreur : compte en attente

```
curl -s -X POST http://localhost:3001/login \  
-H "Content-Type: application/json" \  
-d '{"username":"bob","password":"Pass123;"}'
```

Résultat :

```
{"message":"Votre inscription est en attente  
de validation par un administrateur."}
```

POST /signup — Erreur : champs manquants

```
curl -s -X POST http://localhost:3001/signup \  
-H "Content-Type: application/json" \  
-d '{"username":"bob"}'
```

Résultat :

```
{"message":"Username et mot de passe requis"}
```

GET /users — Liste des utilisateurs validés

```
curl http://localhost:3001/users
```

Résultat :

```
{  
  "users": [  
    {  
      "username": "madina",  
      "firstName": "Madina",  
      "lastName": "LALAOUI",  
      "role": "admin",  
    },  
    {  
      "username": "rasheequa",  
      "firstName": "Rasheequa",  
      "lastName": "Bagadad saib",  
      "role": "member",  
    },  
    {  
      "username": "jade",  
      "firstName": "Jade",  
      "lastName": "DUPONT",  
      "role": "member",  
    },  
    {  
      "username": "anne",  
      "firstName": "Anne",  
      "lastName": "Hartman",  
      "role": "member",  
    }  
  ]  
}
```

GET /users/:username — Erreur : inexistant

```
curl -s http://localhost:3001/users/inconnu
```

Résultat :

```
{"message":"Utilisateur non trouvé"}
```

PUT /users/:username/validate — Valider

```
curl -s -X PUT http://localhost:3001/users/bob/validate
```

Résultat :

```
{"message":"Utilisateur validé"}
```

PUT /users/:username/reject — Rejeter

```
curl -s -X PUT http://localhost:3001/users/bob/reject
```

Résultat :

```
{"message":"Utilisateur rejeté"}
```

PUT /users/:username/role — Erreur : propre rôle

```
curl -s -X PUT http://localhost:3001/users/madina/role \  
-H "Content-Type: application/json" \  
-d '{"role":"member","requester":"madina"}'
```

Résultat :

```
{"message":"Impossible de modifier son propre rôle"}
```

POST /login — Erreur : mot de passe incorrect

```
curl -s -X POST http://localhost:3001/login \  
-H "Content-Type: application/json" \  
-d '{"username":"jade","password":"mauvais"}'
```

Résultat :

```
{"message":"Utilisateur ou mot de passe incorrect"}
```

POST /signup — Créer un compte (pending)

```
curl -s -X POST http://localhost:3001/signup \  
-H "Content-Type: application/json" \  
-d '{"username":"bob","password":"Pass123;",  
"firstName":"Bob","lastName":"Martin"}'
```

Résultat :

```
{"message":"Utilisateur créé"}
```

POST /signup — Erreur : username déjà utilisé

```
curl -s -X POST http://localhost:3001/signup \  
-H "Content-Type: application/json" \  
-d '{"username":"bob","password":"Pass123;",  
"firstName":"Bob","lastName":"Martin"}'
```

Résultat :

```
{"message":"Nom d'utilisateur déjà utilisé"}
```

GET /users/:username — Profil utilisateur

```
curl http://localhost:3001/users/madina
```

Résultat :

```
{"username":"madina","firstName":"Madina",  
"lastName":"LALAOUI","role":"admin"}
```

GET /users/pending — Comptes en attente

```
curl -s http://localhost:3001/users/pending
```

Résultat :

```
{"users": [  
  {  
    "username": "bob",  
    "firstName": "Bob",  
    "lastName": "Martin",  
    "dateCreation": "2026-06-02T13:44:29.330Z"  
  }  
]}
```

PUT /users/:username/validate — Erreur

```
curl -s -X PUT http://localhost:3001/users/inconnu/validate
```

Résultat :

```
{"message":"Utilisateur non trouvé ou déjà validé"}
```

PUT /users/:username/role — Modifier rôle

```
curl -s -X PUT http://localhost:3001/users/rasheequa/role \  
-H "Content-Type: application/json" \  
-d '{"role":"admin","requester":"madina"}'
```

Résultat :

```
{"message":"Rôle mis à jour"}
```

PUT /users/:username/role — Erreur : rôle invalide

```
curl -s -X PUT http://localhost:3001/users/rasheequa/role \  
-H "Content-Type: application/json" \  
-d '{"role":"superuser","requester":"madina"}'
```

Résultat :

```
{"message":"Rôle invalide"}
```